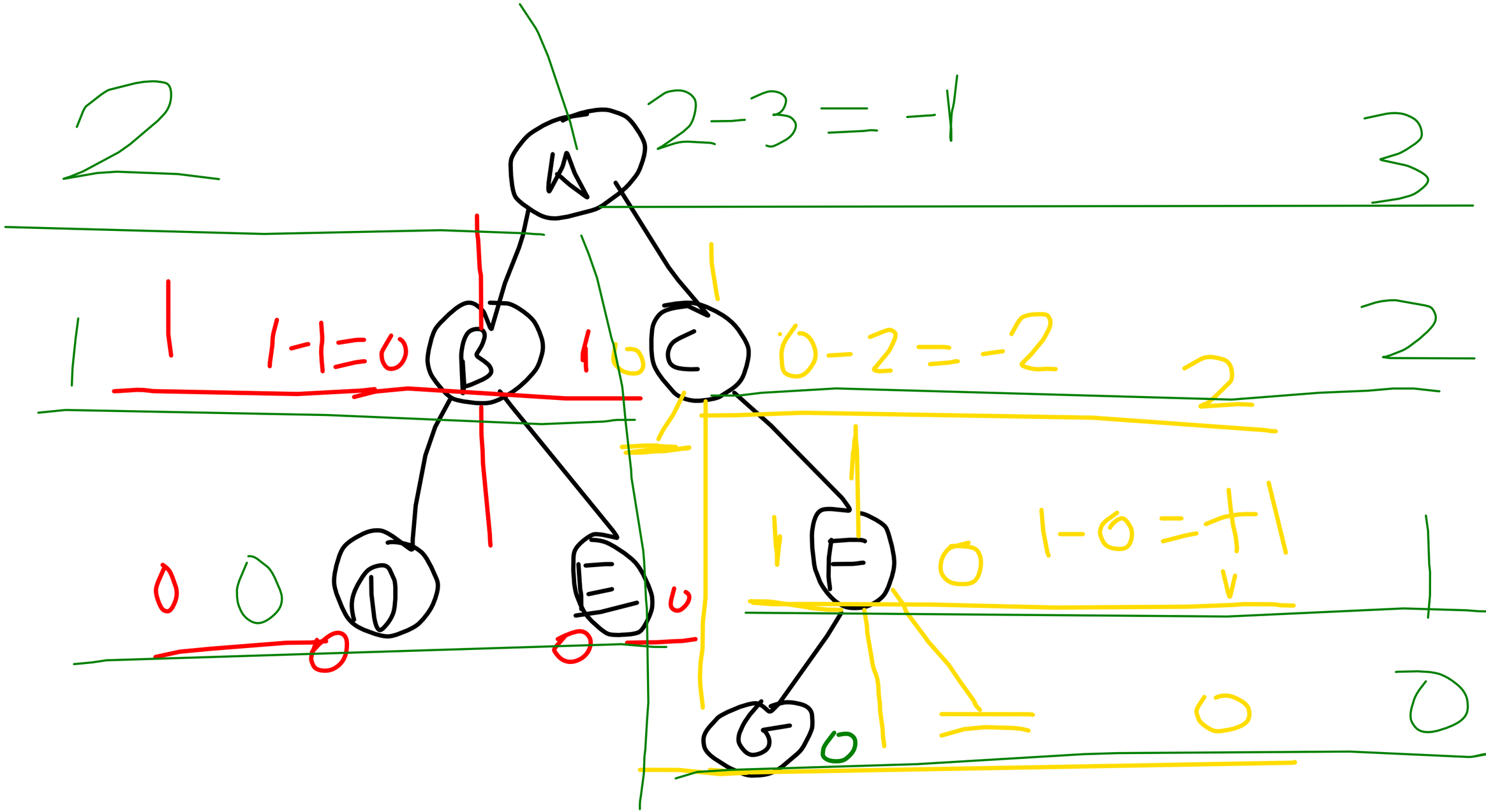
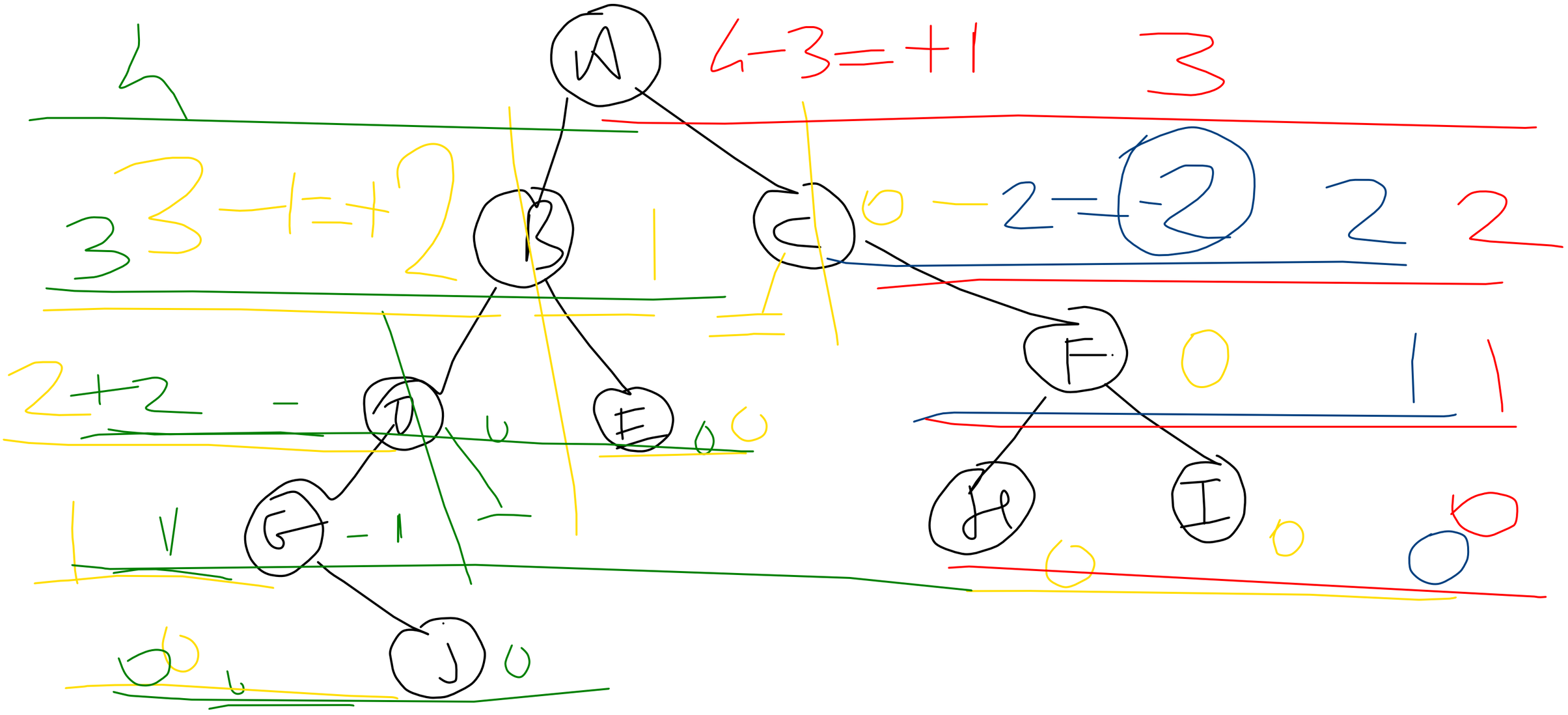
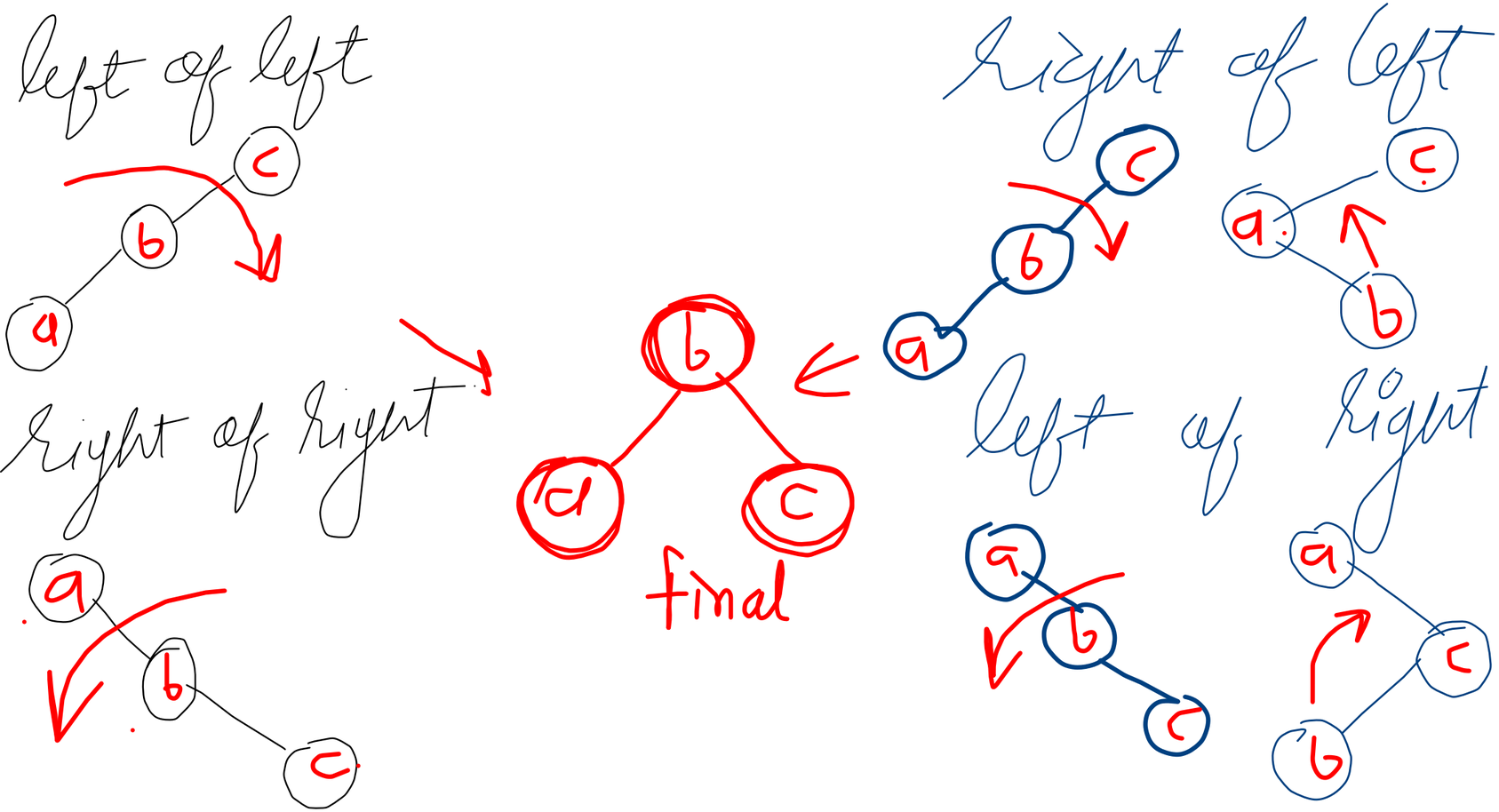
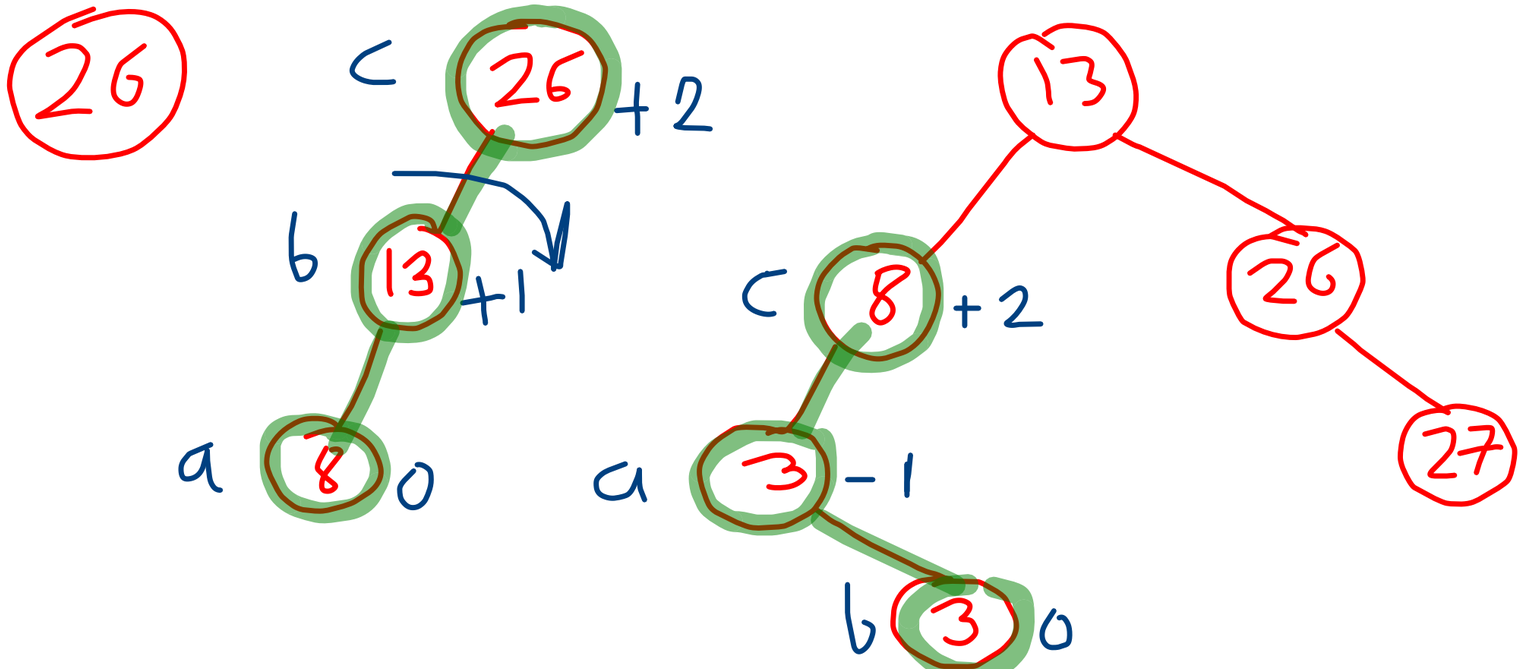

A
B
C
D
E
F
G
2-3=-1
1-1=0
0-2=-2
1-0=+1

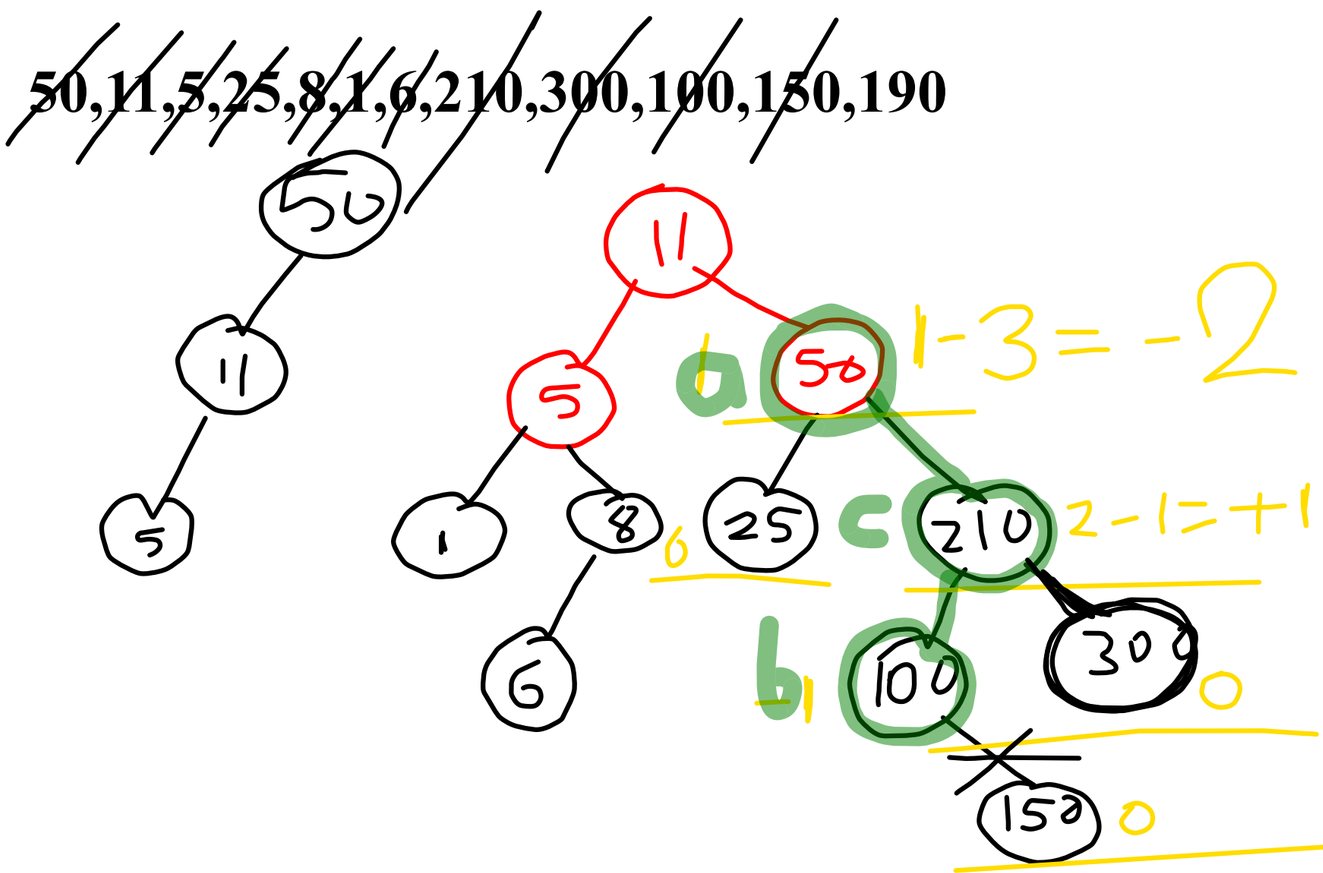

A
B
C
D
E
F
G
H
I
J

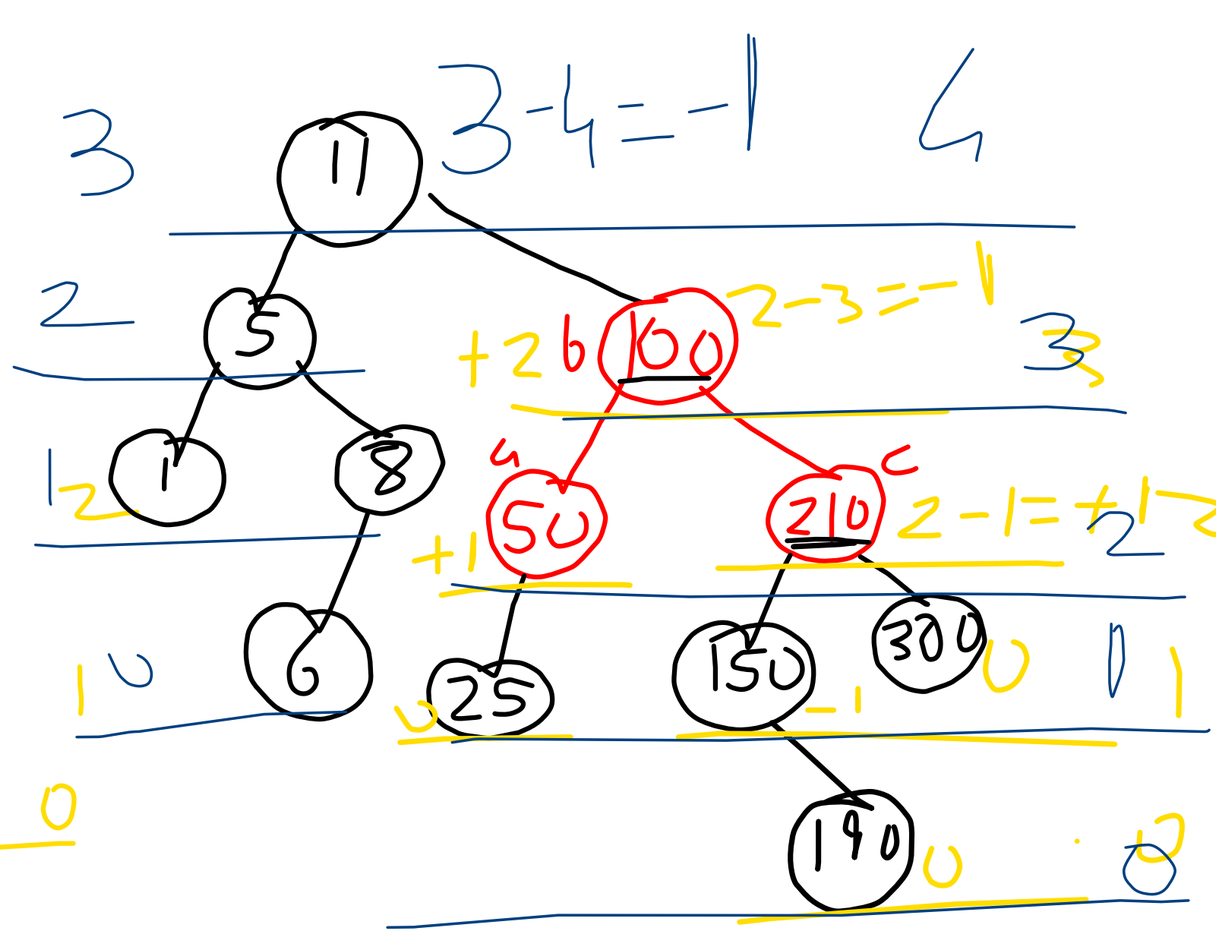
If the balance factor is out of +1 and -1, then we would need to perform rotation.

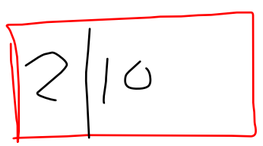


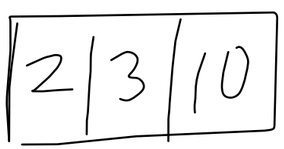

26
c
26
+2
b
13
+1
a
8
0
13
c
8
+2
26
a
3
-1
27
b
3
0
13
3
26
a
3
b
8
27
c

50,11,5,25,8,1,6,210,300,100,150,190

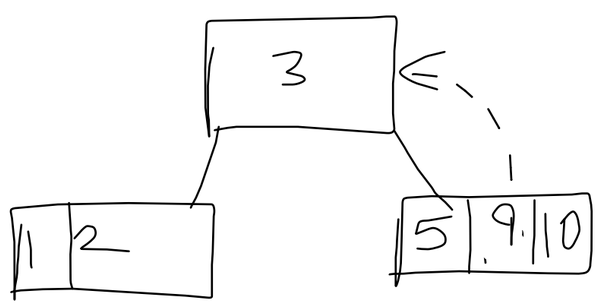








Order



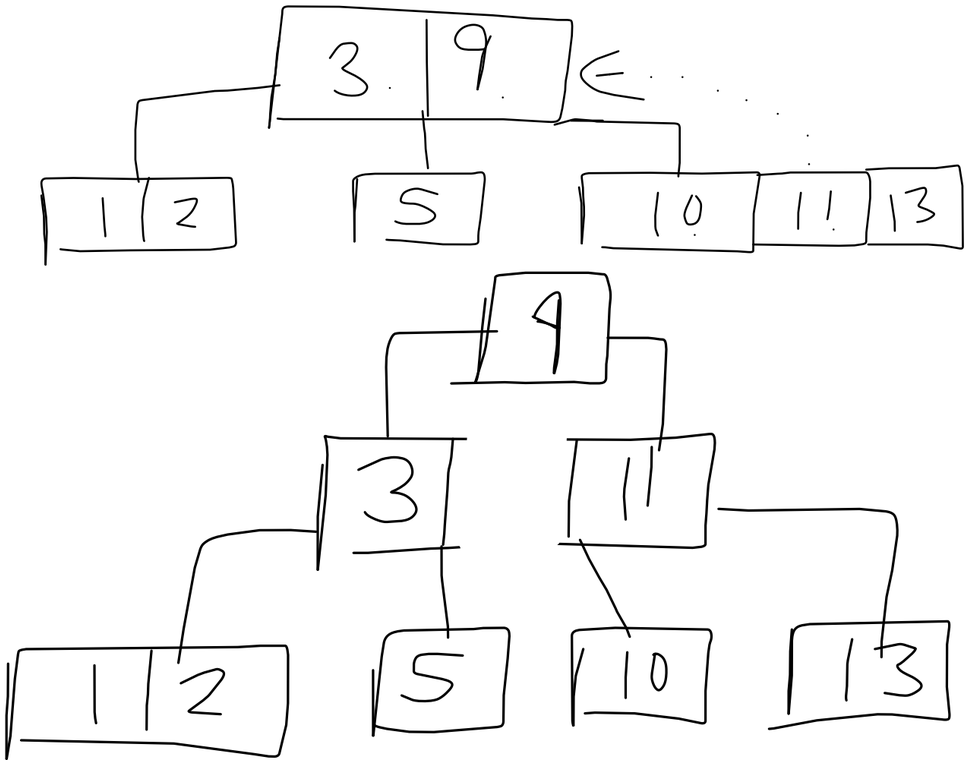
M=3

Max=3-1=2

Min=3/2=1

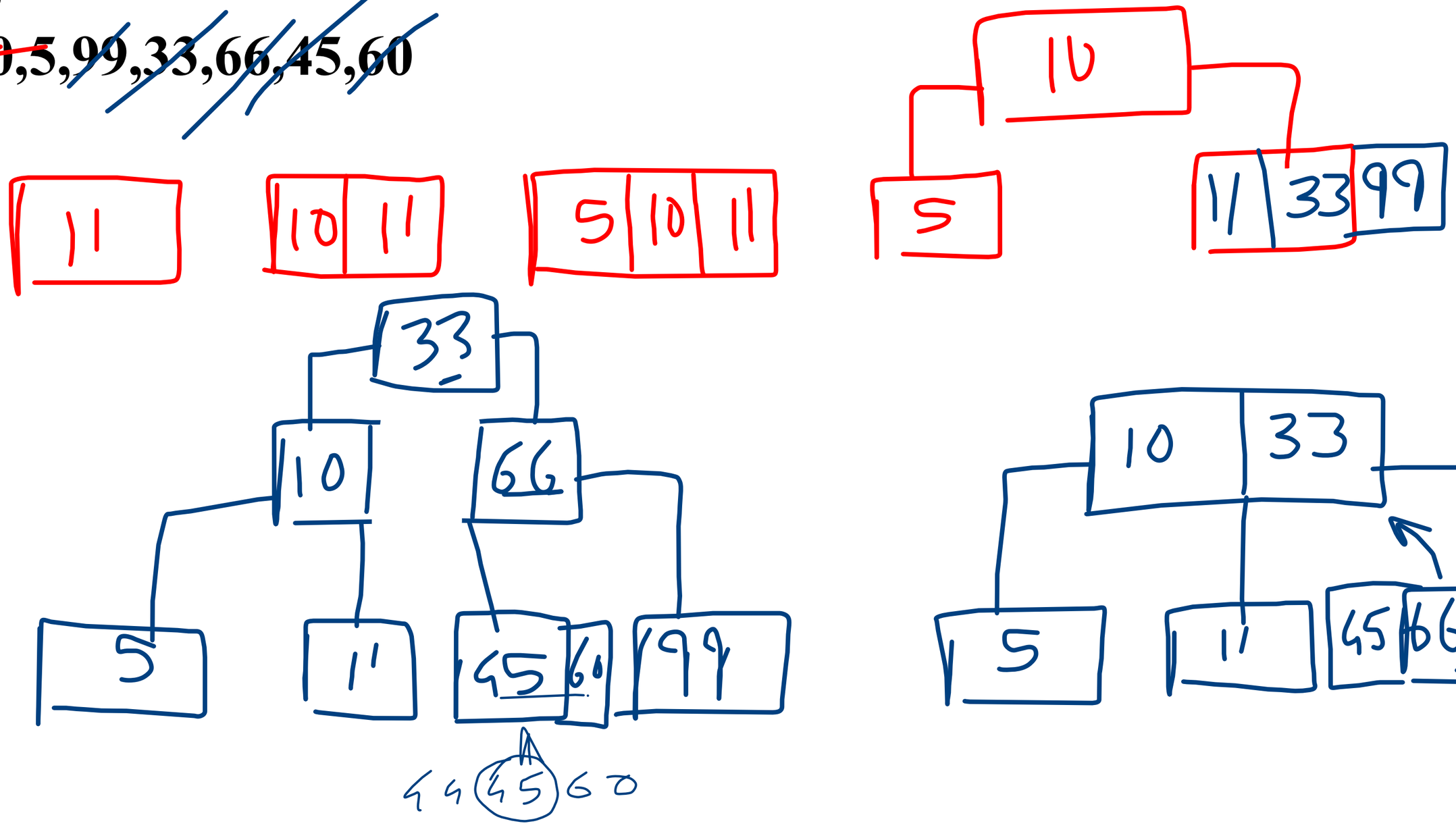
Split=3

10,2,3,1,5,9,11,13



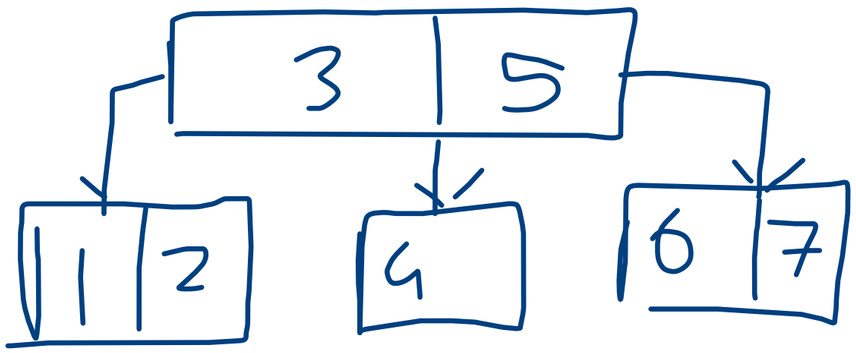
M=3

~~**11,10,5,99,33,66,45,60**~~



B+

The conventional B-tree has a drawback: data is scattered over n nodes. For sequential access we would take more time and incur the overhead of visiting different nodes for access.



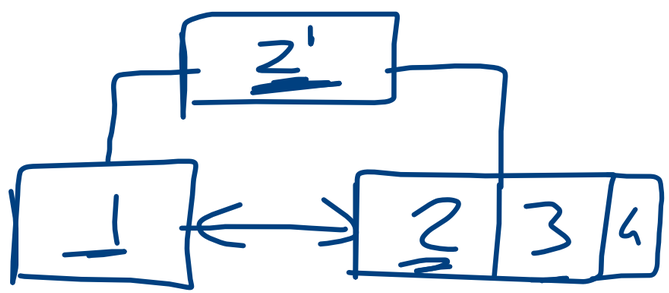
m=3

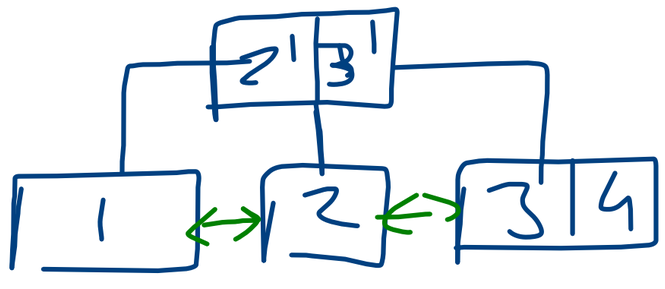
See all

B+ is an advanced form of a B-tree where data is enhanced for sequential access and only indexing-based data is given on split.
All data on leaf node only.
On split data will not be shifted up. Only their index will be shifted up.
All leaf nodes are interconnected to each other via bidirectional links.

M=3







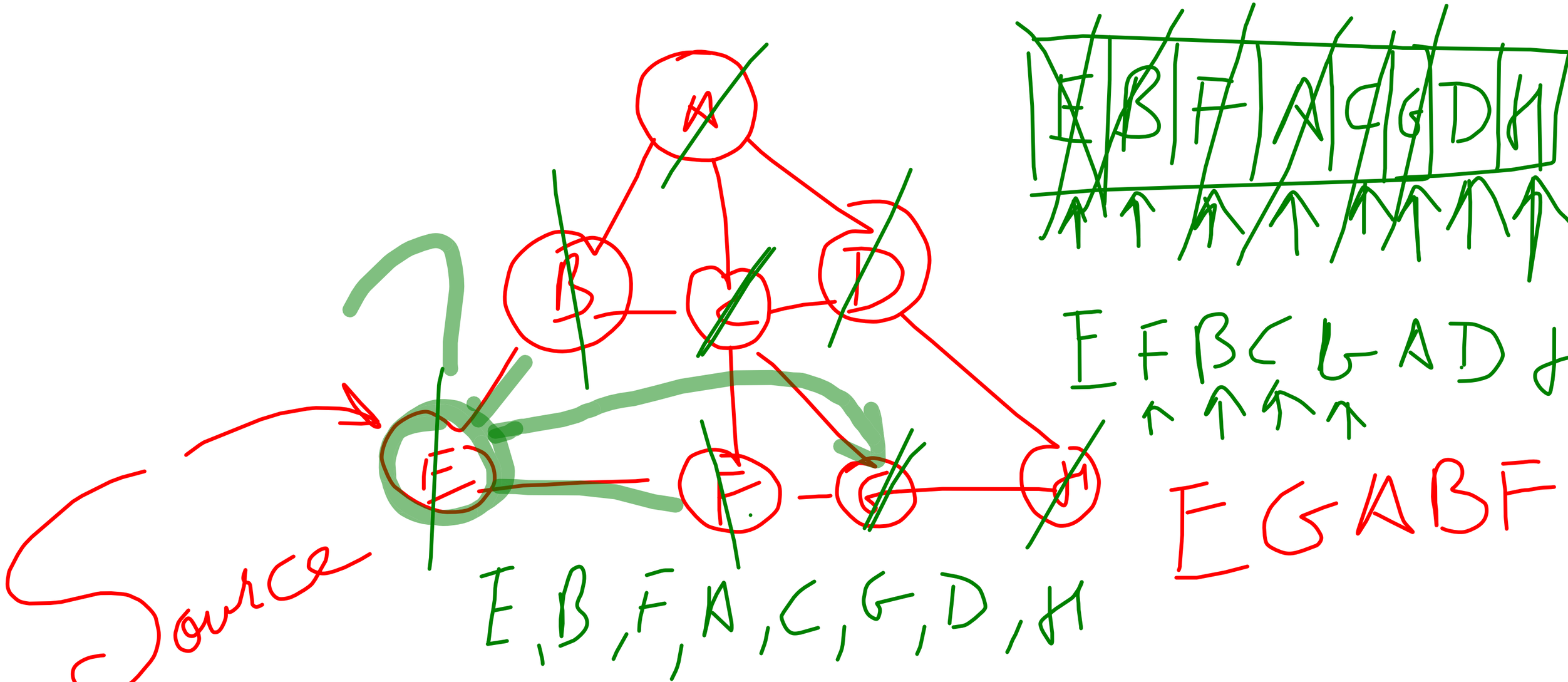
Adjacency matrix

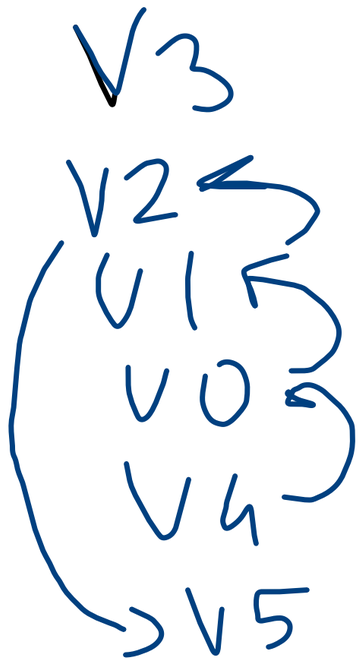
It's a matrix representation, easy to create and faster to access, but comes at the cost of static memory allocation, so it cannot grow or shrink.

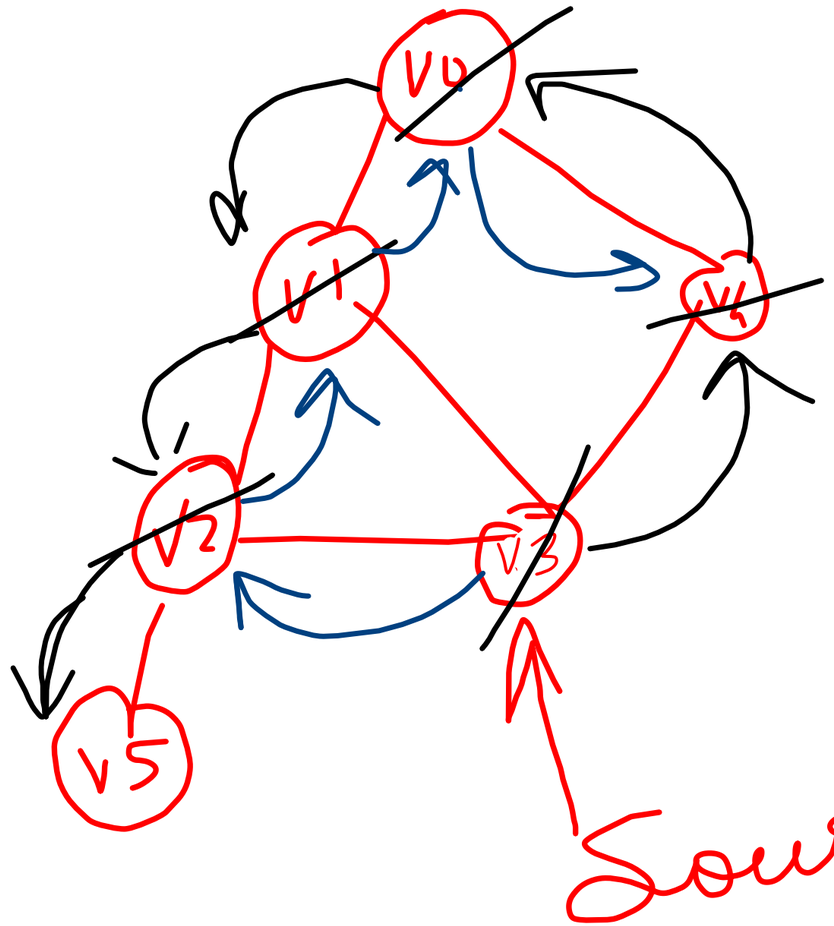
Adjacency List

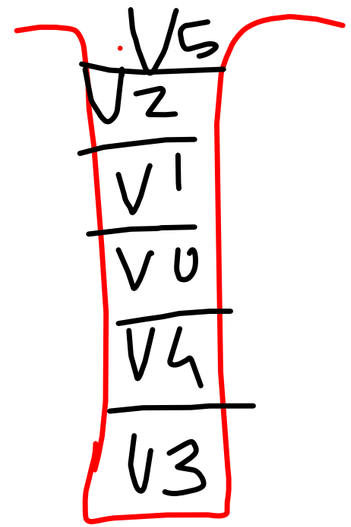
It is a collection of linked lists, dynamic in nature. Only needed data will be assigned. Unneeded data is not even stored. Excellent in memory storage and handling. But very slow and complex to use and maintain, thanks to the multiple linked lists and traversal of linked lists

Graph searching, also called graph traversal
DFS(Depth-first search)
BFS(Breadth-first search)
Both techniques are known as blind searching and do not give a unique answer. There can be multiple answers for DFS and BFS, depending totally on how a user is using the algorithm.


E B F A C G D H
E F B C G A D H
E G A B F
Source
E, B, F, A, C, G, D, H







V3, V4, V0, V1, V2
V5

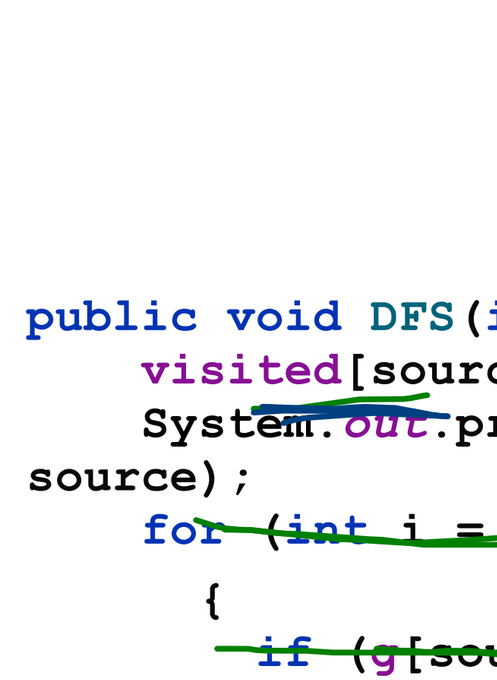
In the current program we are implementing a graph using an adjacency matrix.
Creating a graph would take the number of nodes and the user has to set everything in terms of edge cost. You can give directed, undirected, weighted, or unweighted. Everything depends on the user's input.

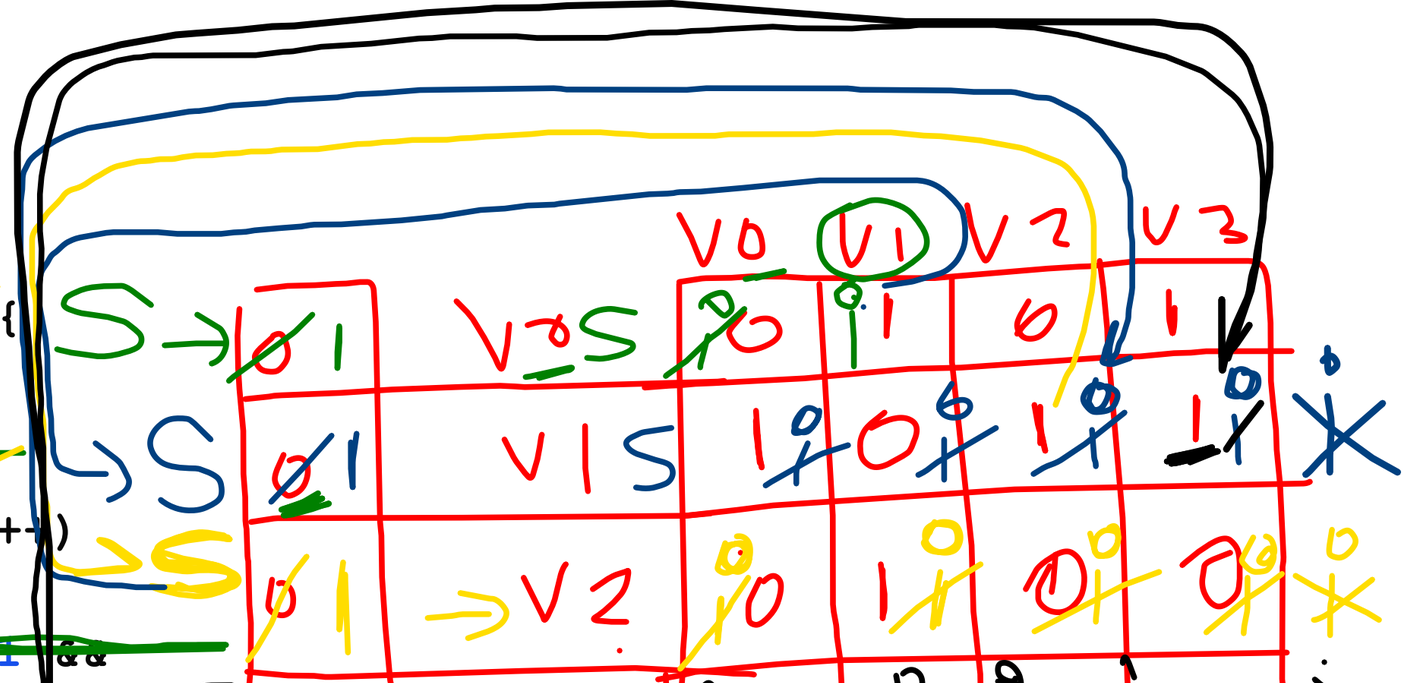
g[][]

In order to detect whether visited or not, we are creating a `visited` array which has `v` entries. `v` is the number of nodes.

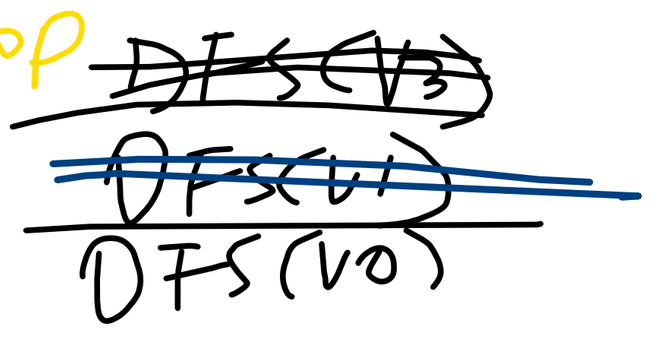
Depending on whether we need to perform a DFS or a BFS, respective algorithms are coded separately.

```
public void DFS(int source){
    visited[source] = 1;
    System.out.println("V" +
source);
    for (int i = 0; i < v; i++)
      {
       if (g[source][i] == 1 &&
visited[i] != 1)
        //neighbour and unvisited
        {
            DFS(i);
        }
    }
}
```



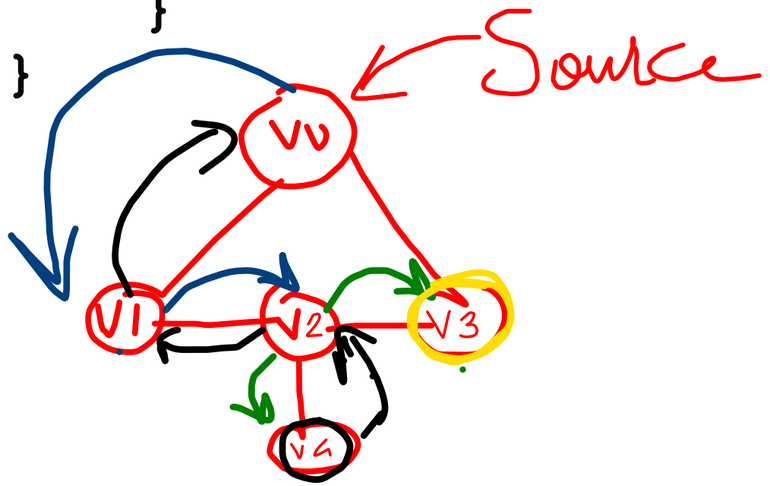








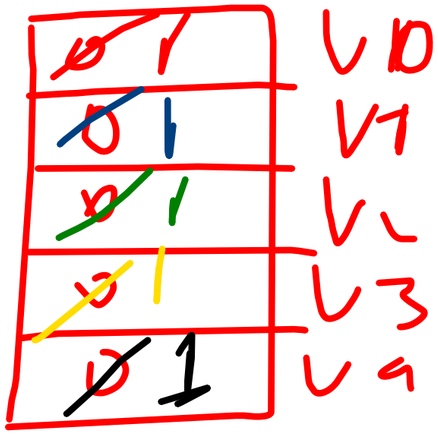
```
public void DFS(int source) {
    visited[source] = 1;
    System.out.println("V" + source);
    for (int i = 0; i < v; i++) {
        if (g[source][i] == 1 &&
visited[i] != 1)
        //neighbour and unvisited
        {
            DFS(i);
        }
    }
}
```



When are we printing on push not on pop.

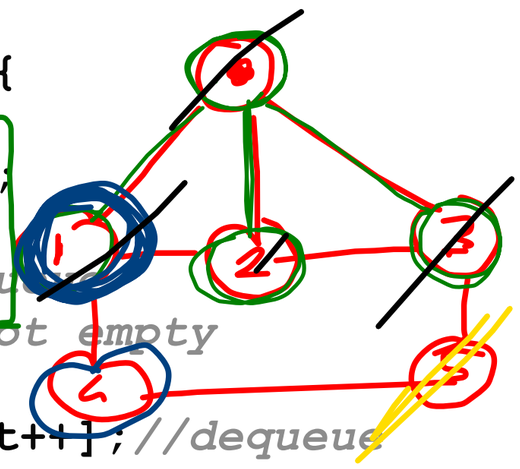
V0 V1 V2 V3 V4

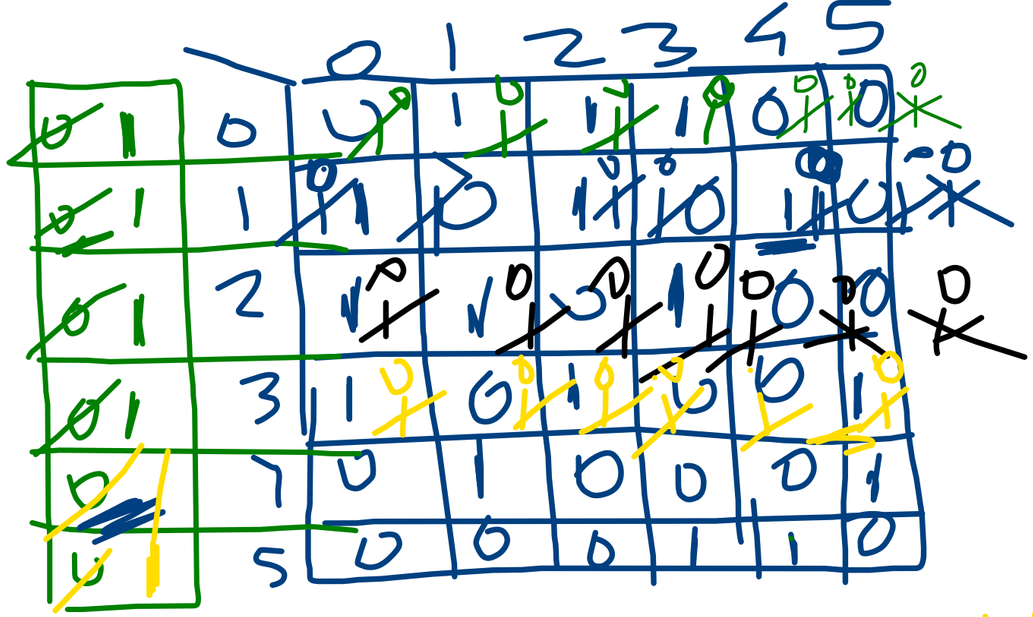
V3 pop

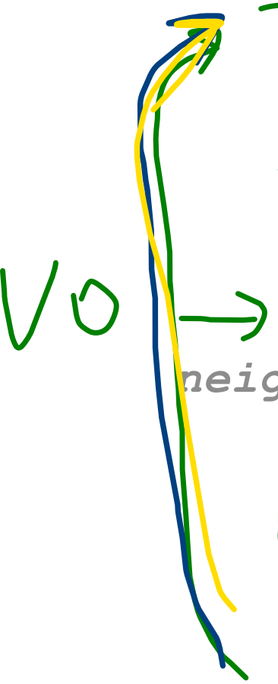


```java
public void BFS(int source) {
    int q[] = new int[v];
    int front = 0, rear = -1;
    visited[source] = 1;
    q[++rear] = source;//enqueue
    while (front <= rear)//not empty
    {
        int element = q[front++];//dequeue
        System.out.print("V" + element + "-");

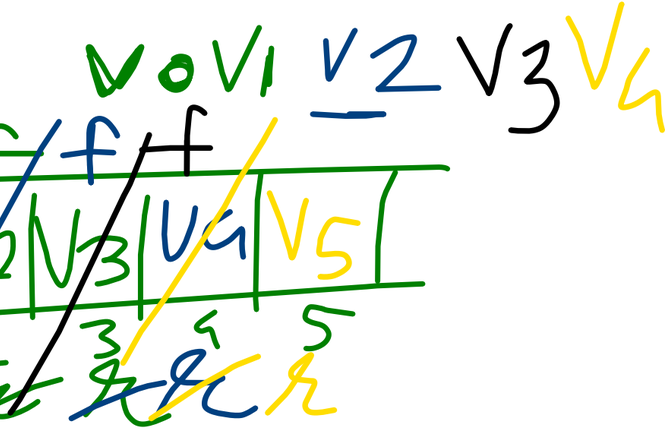
        for (int i = 0; i < v; i++) {
            if (g[element][i] == 1 && visited[i] != 1)//
neighbour and unvisited
            {
                visited[i] = 1;//visited
                q[++rear] = i;//enqueue
            }
        }
    }
}
```







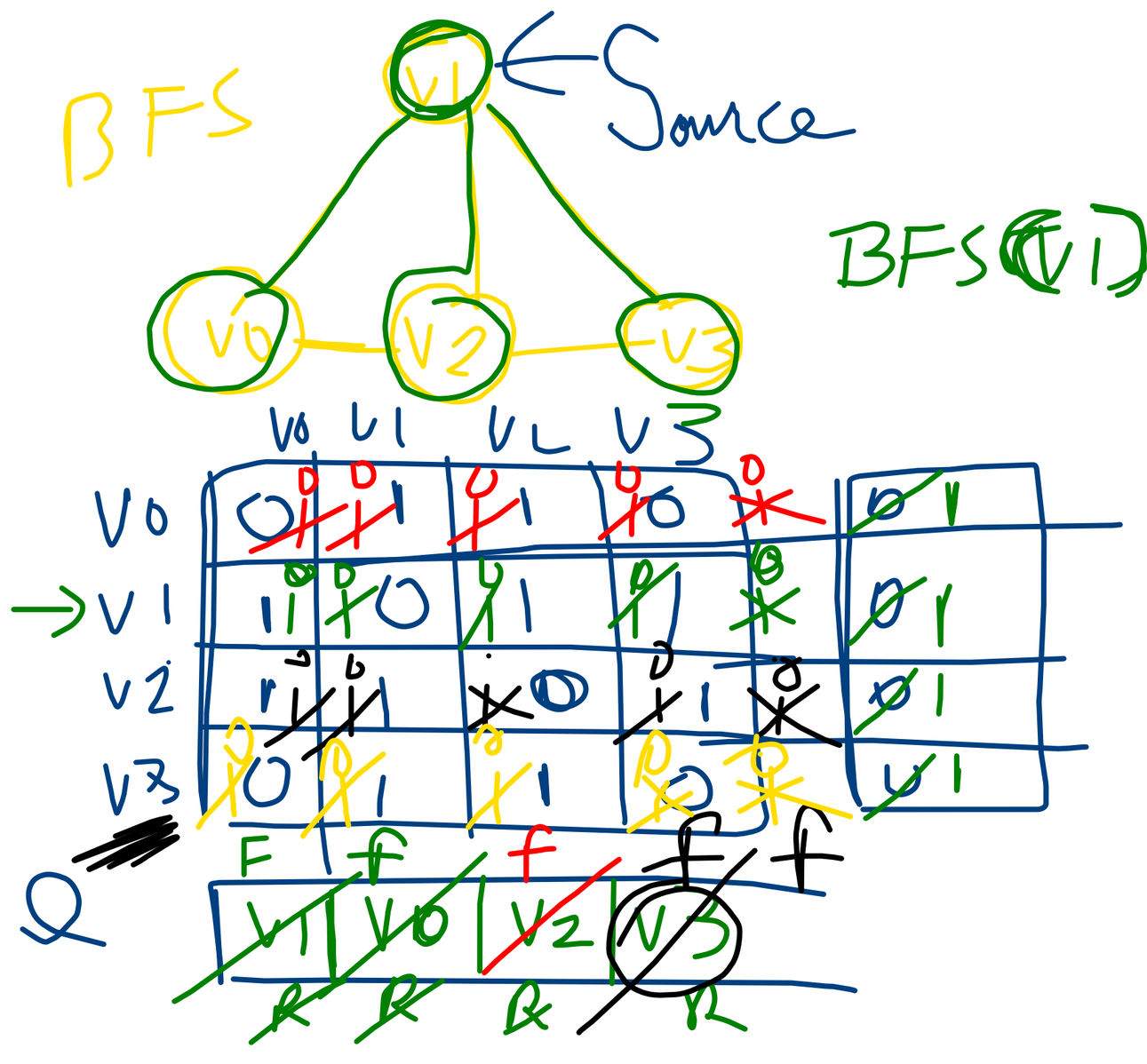


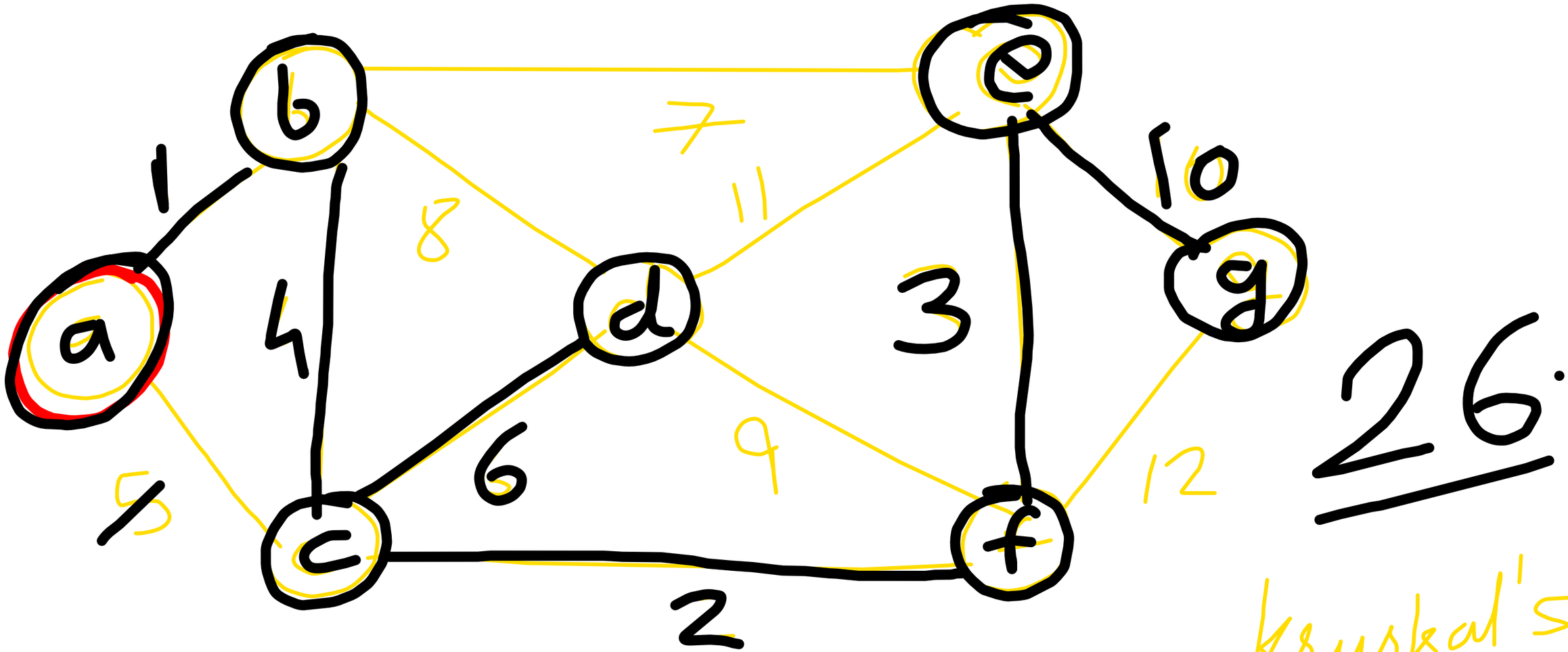


```
public void BFS(int source) {
    int q[] = new int[v];
    int front = 0, rear = -1;
    visited[source] = 1;
    q[++rear] = source;//enqueue
    while (front <= rear)//not empty
    {
        int element = q[front++];//dequeue
        System.out.print("V" + element + "-");

        for (int i = 0; i < v; i++) {
            if (g[element][i] == 1 && visited[i] != 1)//
neighbour and unvisited
            {
                visited[i] = 1;//visited
                q[++rear] = i;//enqueue
            }
        }
    }
}
```

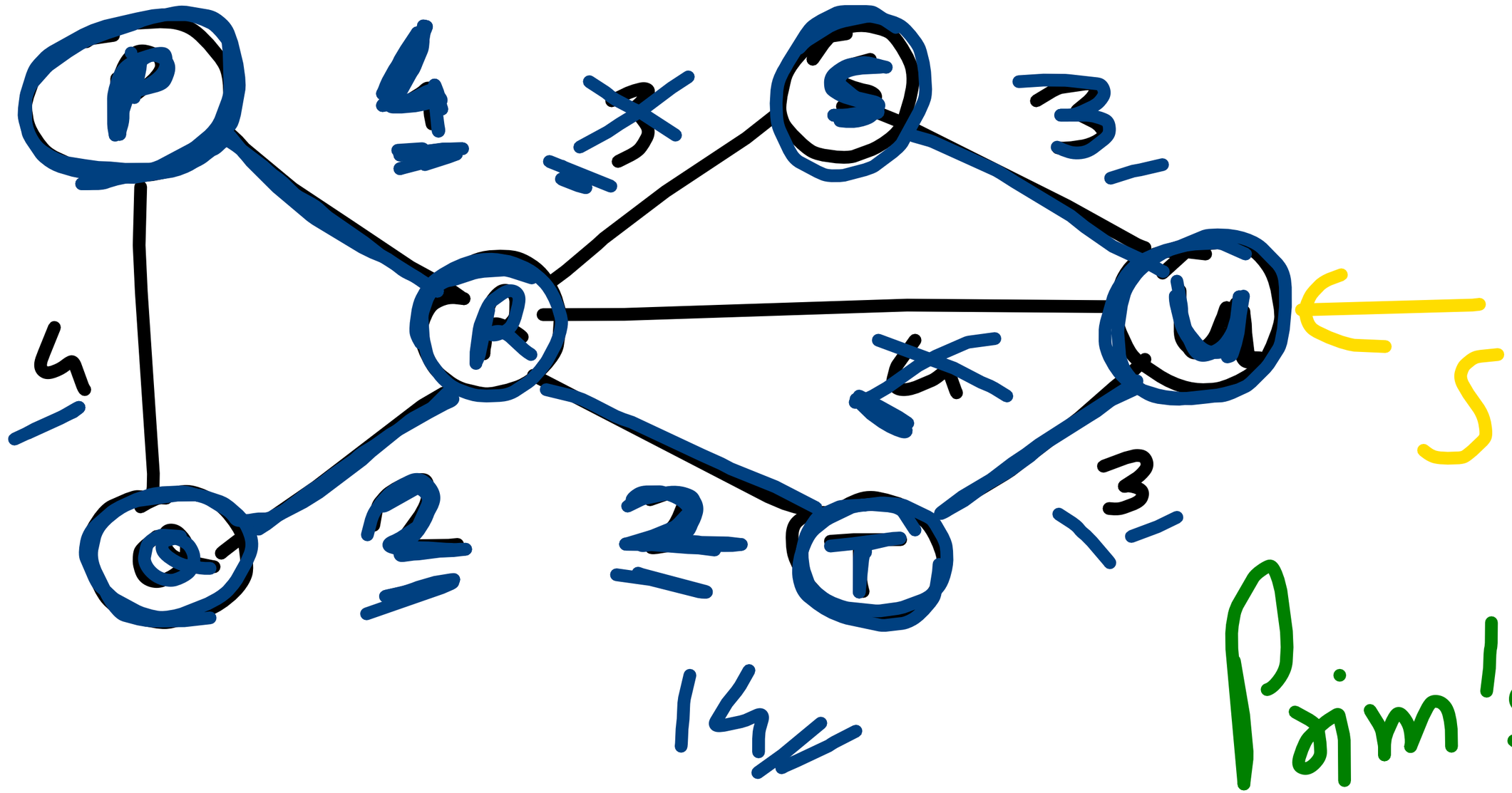
V1 V0 V2 V3





26

kruskal's


P
S
R
U
Q
T
4
7
3
4
2
2
13
14
5
Prim's

1.List all the things which you really love doing.
Then choose the top three(1) things which you ultimately love to do.
2.Ask yourself a question about this line: Can you know everything or are you ready to know everything?
3.Ask your friends and your critics what you are best in and what you are really good at.
4.Figure out how to make money with this.